DESENVOLVIMENTO DE API BACK-END · ENGENHARIA DE SOFTWARE · 6º PERÍODO

Semana 15
Governança e observabilidade

Apostila de estudo: teoria, exemplos, construção com IA e exercícios

A API está no ar. Agora é preciso enxergar o que ela faz, registrar por que ela é assim e mudá-la sem surpreender quem depende dela. Imagine um cliente dizendo que a contratação dele deu erro às 14h: como descobrir o que aconteceu sem reproduzir o erro? Esta apostila responde a essa pergunta com métricas, logs e rastreio, e depois trata das decisões que protegem a qualidade da API ao longo do tempo: características de arquitetura, ADRs, governança e API Gateway.

Conteúdo

1. Objetivos da semana
2. Monitorar e observar
3. Os três pilares
4. Tipos de métrica
5. O que medir primeiro: RED e USE
6. SLI, SLO e SLA
7. Logs estruturados
8. Correlação: trace, span e request id
9. Características de arquitetura
10. Decisões de arquitetura e ADR
11. Governança de API
12. API Gateway
13. Aplicação no ServiceHub
14. Construindo com IA
15. Glossário
16. Exercícios
17. Para estudar

1. Objetivos da semana

↑ conteúdo

Ao final desta semana você deve ser capaz de:

1. diferenciar monitoramento de observabilidade e explicar o papel de logs, métricas e traces;
2. escolher o tipo de métrica certo (contador, gauge, histograma) e aplicar RED, USE, SLI, SLO e SLA;
3. explicar como logs estruturados e um id por requisição permitem investigar um erro sem reproduzi-lo;
4. identificar características de arquitetura, propor como medi-las e registrar decisões em ADRs;
5. descrever o que é governança de API e o papel de um API Gateway.

2. Monitorar e observar

↑ conteúdo

Monitoramento acompanha sinais escolhidos de antemão e avisa quando passam de um limite: "a taxa de erro passou de 5%", "o disco está 90% cheio". **Observabilidade** é a capacidade de entender o estado interno de um sistema a partir dos dados que ele emite, inclusive para perguntas que ninguém previu quando o sistema foi escrito.

Critério	Monitoramento	Observabilidade
Pergunta típica	Está funcionando?	Por que não está funcionando?
Parte de	Painéis e alertas definidos antes	Dados ricos, explorados depois
Bom para	Falhas conhecidas: disco cheio, serviço fora do ar	Falhas novas, que ninguém previu
Exemplo	Alerta quando a taxa de erro passa do limite	Filtrar os erros por rota, versão e cliente

Uma não substitui a outra: **o monitoramento avisa, a observabilidade explica**. Um alerta diria que houve erros às 14h; só dados com contexto dizem qual cliente, qual rota e por quê.

CONFUSÃO COMUM

Um painel bonito não é observabilidade. Se os dados por trás dele não permitem filtrar e cruzar informações para responder a uma pergunta nova, o painel só mostra o que alguém já sabia que queria ver.

3. Os três pilares: logs, métricas e traces

↑ conteúdo

Os dados que um sistema emite costumam ser agrupados em três tipos, chamados de pilares da observabilidade. Os exemplos abaixo são de uma loja virtual genérica.

Pilar	O que é	Responde	Exemplo
Logs	Registro de um evento, com horário, nível e contexto. Detalhado e caro de guardar em volume.	O que aconteceu, exatamente?	`ERROR pagamento recusado pedido=981`
Métricas	Números agregados ao longo do tempo. Baratos de guardar e bons para alertas.	Quanto, com que frequência, está piorando?	Requisições por segundo, memória em uso
Traces	O caminho de uma requisição por todos os componentes, com o tempo gasto em cada um.	Onde a requisição demorou ou falhou?	checkout → estoque → banco

A diferença de custo explica como eles são usados juntos. Uma métrica é um número por intervalo de tempo, não importa quantas requisições ocorreram; um log é uma linha por evento. Por isso **se alerta em métrica e se investiga em log**: as métricas avisam que algo mudou, os traces mostram onde, os logs contam o que aconteceu.

Há ainda um quarto sinal, mais simples: a **saúde** (*health*), que responde só "está de pé?". É o que o Docker, o CI e a plataforma de hospedagem consultam, como visto na Semana 14.

4. Tipos de métrica

↑ conteúdo

Tipo	O que mede	Comportamento	Exemplo genérico
Contador	Quantas vezes algo aconteceu.	Só cresce; zera ao reiniciar a aplicação.	Pedidos criados, respostas 500.
Gauge	Um valor no instante da leitura.	Sobe e desce.	Conexões abertas, itens na fila, memória em uso.
Histograma	Como os valores se distribuem em faixas (*buckets*).	Conta observações por faixa.	Tempo de resposta: quantas abaixo de 100 ms, de 500 ms, de 1 s.

Por que o contador "zerar ao reiniciar" não é problema? Porque o sistema de métricas não olha o valor absoluto, e sim quanto ele cresceu num intervalo (a **taxa**). Se a aplicação reinicia, a ferramenta percebe a queda e continua o cálculo.

Média esconde, percentil mostra

Imagine 100 requisições: 95 levam 50 ms e 5 levam 4 segundos. A média é cerca de 250 ms, um número que nenhum usuário viveu. O **percentil 95 (p95)** é o tempo abaixo do qual ficam 95% das requisições; o p99, 99%. Os percentis altos mostram o que o usuário mais lento sente, e é por isso que tempos de resposta se medem com histograma.

> **CONFUSÃO COMUM**
> O número de pedidos *pendentes agora* é um gauge, porque sobe e desce. O número de pedidos *criados* é um contador. Usar gauge para algo que deveria ser contador faz perder eventos que acontecem entre duas coletas.

No Micrometer, a biblioteca de métricas que o Spring Boot usa, os tipos se chamam `Counter`, `Gauge` e `Timer`. O `Timer` registra tempos e pode publicar histograma.

5. O que medir primeiro: RED e USE

↑ conteúdo

Medir tudo é caro e confuso. Duas listas mínimas ajudam a começar pelo que importa:

RED · para serviços que atendem requisições

Rate (taxa): quantas requisições por segundo.
Errors (erros): quantas falham.
Duration (duração): quanto tempo levam, em percentis.

USE · para recursos: CPU, memória, pool

Utilization (utilização): quanto do recurso está ocupado.
Saturation (saturação): quanto trabalho espera na fila.
Errors (erros): falhas do próprio recurso.

Numa API, aplica-se RED a cada rota, vista de fora, e USE à JVM e ao pool de conexões com o banco, vistos de dentro. Exemplo de saturação: um pool com 10 conexões, todas em uso, e requisições esperando. A utilização está em 100%, mas é a fila que explica a lentidão. No Spring Boot, o Actuator já entrega RED por rota na métrica `http.server.requests`.

6. SLI, SLO e SLA: medir, prometer, contratar

↑ conteúdo

Sigla	O que é	Exemplo (números ilustrativos)
SLI · indicador	Uma medida do serviço do ponto de vista do usuário.	Fração das requisições respondidas com sucesso em menos de 300 ms.
SLO · objetivo	A meta interna que a equipe quer manter para o SLI num período.	99,5% das requisições dentro do SLI, medido em 30 dias.

Sigla	O que é	Exemplo (números ilustrativos)
SLA · acordo	Compromisso com o cliente, com consequência se for descumprido.	Abaixo de 99% no mês, o cliente recebe crédito na fatura.

Os números da tabela são **exemplos**, não recomendações: cada serviço escolhe os seus. Repare que o SLO é mais rígido que o SLA. A equipe precisa perceber o problema e reagir antes de quebrar o contrato.

Orçamento de erro é a folga que o SLO deixa. Com um SLO de 99,5%, até 0,5% das requisições podem falhar no período. Enquanto há folga, a equipe pode arriscar entregas novas; gastou a folga, a prioridade passa a ser estabilidade. Assim, confiabilidade vira uma decisão com número, e não uma discussão de opinião.

NO SERVICEHUB

Um SLI que faz sentido: a fração dos `POST /api/v1/service-requests` (contratações) respondidos sem erro 5xx. É a operação que gera receita para a plataforma.

7. Logs estruturados

↑ conteúdo

Um log sem contexto é uma frase solta no meio de milhares. **Log estruturado** é um evento escrito como pares chave-valor, em geral em JSON, para que a ferramenta de logs filtre por campo em vez de procurar texto solto.

Texto livre

```
14:02:11 ERROR Falha ao criar pedido
  do cliente 77 na rota /orders
```

Achar todos os erros do cliente 77 exige uma expressão regular frágil, que quebra quando alguém muda a frase.

Estruturado (JSON)

```
{"@timestamp":"…","log.level":"ERROR",
 "message":"Falha ao criar pedido",
 "customerId":77,"route":"/orders"}
```

A busca é um filtro: `log.level=ERROR` e `customerId=77`.

Para que ferramentas diferentes entendam os mesmos campos, existem esquemas de nomes padronizados. Um deles é o **ECS** (*Elastic Common Schema*), com campos como `@timestamp`, `log.level` e `message`. O Spring Boot escreve logs nesse formato com uma propriedade, sem biblioteca extra.

Níveis de log

Nível	Quando usar	Exemplo genérico
ERROR	Falha que alguém precisa olhar.	Banco indisponível ao gravar pedido.

Nível	Quando usar	Exemplo genérico
WARN	Algo estranho, mas o sistema seguiu.	Nova tentativa de chamada ao fornecedor.
INFO	Fluxo normal relevante para o negócio.	Pedido 981 criado.
DEBUG	Detalhe para depurar, em geral só em desenvolvimento.	SQL gerado e parâmetros.

ERROS COMUNS EM CÓDIGO GERADO POR IA

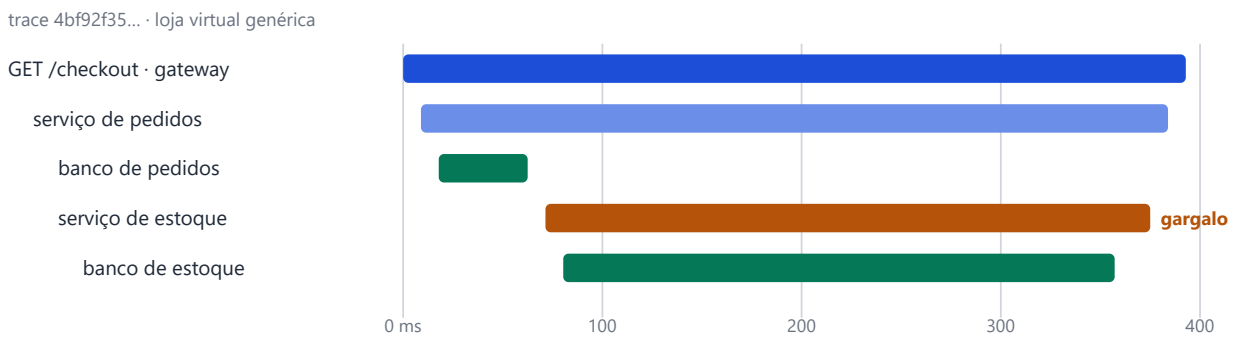
Registrar como ERROR uma situação esperada, como um 404 de recurso inexistente, enche o alerta de ruído: erro do cliente é WARN ou INFO. E log também é dado a proteger: nada de senha, token JWT ou documento pessoal completo, nem "só para depurar". Também não use `System.out.println`: ele não tem nível, não tem contexto e não sai em JSON.

8. Correlação: trace, span e request id

↑ conteúdo

Num sistema com vários serviços, uma requisição do usuário vira várias chamadas internas. O **rastreamento distribuído** (*distributed tracing*) registra essa viagem.

- **Trace:** a viagem completa de uma requisição, identificada por um *trace id*.
- **Span:** uma etapa da viagem, com nome, início, duração e o span pai que a chamou.
- **Correlação:** o id viaja num header de um serviço para o outro, e todo log e span carrega esse id.



Um trace em cascata: cada barra é um span, o recuo mostra quem chamou quem e o comprimento mostra a duração. Aqui o estoque é o gargalo.

Entre serviços, o padrão **W3C Trace Context** leva o id no header `traceparent`. Uma aplicação isolada não precisa de tudo isso, mas precisa da ideia central: **um id por requisição, carimbado em cada linha de log**. É o papel do header `X-Request-Id`: o cliente pode enviá-lo ou a API gera um; a API o devolve na resposta; quem reporta um problema informa o id, e você filtra os logs por ele.

MDC: o id em toda linha

Para que cada linha de log carregue o id sem que cada método o receba como parâmetro, as bibliotecas de log Java oferecem o **MDC** (*Mapped Diagnostic Context*): um mapa ligado à thread que atende a requisição. Um filtro coloca o id no MDC no início da requisição; o padrão de log ou o formato JSON o inclui em cada linha; no fim, o filtro remove o id.

POR QUE REMOVER NO FIM

Servidores web reaproveitam threads de um pool. Se o id não for removido, a próxima requisição atendida pela mesma thread herda o id da anterior, e os logs de uma aparecem com o id da outra: um defeito difícil de achar justamente na ferramenta de achar defeitos.

9. Características de arquitetura

↑ conteúdo

O que a API faz é o domínio. *Como* ela precisa ser é arquitetura. Em Richards e Ford, uma **característica de arquitetura** é um requisito que não é do domínio, influencia a estrutura do sistema e é crítico para o sucesso dele. Elas costumam ser agrupadas assim:

Grupo	O que cobre	Exemplos
Operacionais	Como o sistema se comporta rodando.	Disponibilidade, desempenho, escalabilidade, recuperação.
Estruturais	Como o código é organizado.	Manutenibilidade, extensibilidade, portabilidade, configurabilidade.
Transversais	O que atravessa tudo.	Segurança, privacidade, acessibilidade, exigências legais.

Para identificá-las, leia as preocupações do negócio e traduza: "a loja não pode cair na Black Friday" vira escalabilidade e disponibilidade. E escolha **poucas**. Querer todas no máximo produz uma arquitetura genérica, que não atende bem nenhuma; listar dez características "importantes" é o mesmo que não escolher.

Medir e governar

Uma característica que não se mede degrada sem ninguém perceber. As medidas podem ser de três tipos:

Medida	O que observa	Exemplo genérico	No ServiceHub
Operacional	O sistema rodando	p95 de latência, disponibilidade no mês	`http.server.requests` no Actuator
Estrutural	O código	Complexidade ciclomática, dependências entre pacotes	Camadas controller → service → repository

Medida	O que observa	Exemplo genérico	No ServiceHub
De processo	Como a equipe entrega	Cobertura de testes, tempo até o deploy	JaCoCo no `mvnw verify`, CI verde

Governar uma característica é impedir que ela se degrade. A ferramenta para isso é a **função de aptidão** (*fitness function*): uma verificação automática e objetiva de uma característica, que roda no build. Exemplo genérico: um teste que falha se o pacote `controller` importar algo do pacote `repository`, protegendo a separação em camadas.

10. Decisões de arquitetura e ADR

↑ conteúdo

Uma decisão é **arquiteturalmente significativa** quando afeta a estrutura, as características, as dependências, as interfaces ou as técnicas de construção do sistema. Escolher entre H2 e PostgreSQL nos testes é uma delas; o nome de uma variável local, não. Richards e Ford descrevem três antipadrões no modo de decidir:

Antipadrão	O que acontece	Remédio
Proteger-se	A decisão é adiada por medo de errar.	Decidir no último momento responsável, com o que se sabe.
Dia da Marmota	A mesma discussão volta sempre, porque ninguém registrou o porquê.	Justificar a decisão por escrito.
Decisão por e-mail	A decisão se perde em conversas e mensagens.	Um único lugar de registro, e aviso a quem é afetado.

O **ADR** (*Architecture Decision Record*, registro de decisão de arquitetura) resolve os três. No formato proposto por Michael Nygard, ele cabe numa página e fica versionado junto com o código:

Seção	Conteúdo
Título	Número e nome curto: "ADR-0004 — Formato único de erro".
Status	Proposto, aceito ou substituído (por qual ADR), com data.
Contexto	O problema e as forças que obrigaram a decidir, com os fatos da época.
Decisão	O que foi decidido, em voz ativa e sem rodeio.
Consequências	O que fica mais fácil e o que fica mais difícil, incluindo os custos.

Um ADR aceito não se edita: se a decisão muda, escreve-se um novo que substitui o antigo, e o histórico fica. O livro de Richards e Ford propõe seções extras, como *conformidade* (como verificar que a decisão é seguida), que combina bem com as funções de aptidão.

PARA LEMBRAR

O ADR é o "problema, decisão, efeito" de uma escolha. Ele serve a quem entra no projeto depois, ao seu eu de daqui a seis meses e também a um assistente de IA, que passa a ter o contexto para não desfazer uma decisão já tomada.

11. Governança de API

↑ conteúdo

Governança de API é o conjunto de regras e práticas que mantém muitas APIs consistentes, seguras e fáceis de encontrar, sem depender de quem escreveu cada uma. Numa empresa com dezenas de APIs e sem governança, cada equipe inventa o próprio formato de erro e a própria paginação, e quem consome sofre. Não é burocracia: é o mínimo para outra pessoa entender e usar a API sem perguntar ao autor.

Padrões

Um guia de estilo: nomes de rota, versão, formato de erro, paginação, autenticação.

Revisão de design

O contrato é revisado antes do código, por pessoas e por um verificador automático de OpenAPI no CI.

Catálogo

O inventário: cada API com dono, versão, contrato OpenAPI e situação (ativa ou depreciada).

Ciclo de vida

Política de versionamento e de depreciação, vista na Semana 14: anunciar, marcar, conviver e só então remover.

Governança e observabilidade se encontram na depreciação. Sem métrica por rota, decidir quando desligar uma versão antiga é um chute; com `http.server.requests` separado por URI, a equipe sabe quem ainda chama a v1 e pode avisar antes.

12. API Gateway

↑ conteúdo

Um **API Gateway** é um ponto único de entrada entre os clientes e as APIs. Ele aplica, num lugar só, regras que seriam repetidas em cada serviço. As três funções centrais:

Função	O que faz
Roteamento	Decide o destino de cada requisição por caminho, versão ou host: `/v1` para o serviço antigo, `/v2` para o novo. É onde a política de versão vira regra.
Autenticação	Valida o token na borda e repassa a identidade ao serviço.

Função	O que faz
Limite de taxa	*Rate limiting*: no máximo N requisições por cliente numa janela de tempo. Passou do limite, responde `429 Too Many Requests`, em geral com o header `Retry-After`.

Outras funções frequentes: terminação TLS, CORS, logs e métricas de borda. Exemplos de produtos são o Kong, o Spring Cloud Gateway e os gateways gerenciados das nuvens; nesta disciplina o gateway é só conceito.

CONFUSÃO COMUM

O gateway autentica (quem é você), mas não substitui a autorização por recurso. A regra "esta ordem é sua?" depende de dados que só o serviço tem; a proteção contra acesso a objetos de outros usuários (BOLA, Semana 9) continua no service.

No ServiceHub, essas funções estão dentro da própria API: `SecurityConfig`, CORS e `RequestIdFilter`. Com muitas APIs, elas sobem para o gateway e ficam iguais para todas. O ServiceHub não tem limite de taxa; numa API pública, essa seria a primeira barreira contra abuso e força bruta no login.

13. Aplicação no ServiceHub

↑ conteúdo

Actuator: o que expor e para quem

O `pom.xml` tem `spring-boot-starter-actuator` e `micrometer-registry-prometheus`. A exposição na web é uma lista permitida, no `application.properties`:

```
management.endpoints.web.exposure.include=health,info,metrics,prometheus
management.endpoint.health.show-details=when-authorized
management.endpoint.health.roles=ADMIN
```

E o `security/SecurityConfig` decide quem acessa:

```
.requestMatchers("/actuator/health", "/actuator/health/**", "/actuator/info").permitAll()
.requestMatchers("/actuator/**").hasRole("ADMIN")
```

Saúde e info são públicas, para o Docker e a plataforma de hospedagem. Métricas e detalhes da saúde, só ADMIN. Endpoints como `env` e `heapdump` ficam fora da lista porque poderiam vazar configuração e memória.

Uma métrica de negócio

CPU, memória e `http.server.requests` o Micrometer mede sozinho. A métrica que só quem conhece o domínio sabe criar é a de negócio. Em `service/ServiceRequestService` (resumido):

```
public static final String TRANSITIONS_METRIC = "servicehub.service_requests.transitions";
private final MeterRegistry meterRegistry;   // recebido pelo construtor

private void count(RequestStatus to) {
    meterRegistry.counter(TRANSITIONS_METRIC, "to", to.name()).increment();
}
```

`count(RequestStatus.PENDING)` é chamado ao criar a ordem, e `count(target)` a cada mudança de status válida em `changeStatus`, depois de gravar. A tag `to` separa a contagem pelo estado de destino e tem só cinco valores possíveis, os estados de `RequestStatus`: baixa cardinalidade, de propósito. É um contador, porque só cresce.

No `/actuator/prometheus`, o ponto vira sublinhado e o contador ganha o sufixo `_total`. Uma linha da saída (valor ilustrativo) e uma consulta PromQL que mostra a taxa por segundo de cada transição nos últimos 5 minutos:

```
servicehub_service_requests_transitions_total{to="CANCELLED"} 4.0

sum by (to) (rate(servicehub_service_requests_transitions_total[5m]))
```

O **Prometheus** coleta esse endpoint de tempos em tempos e guarda as séries; um painel (por exemplo, no Grafana) e alertas leem dele. Uma alta de `CANCELLED` é um alerta de negócio, não de máquina.

A métrica tem teste

`ObservabilityIntegrationTest` cria uma ordem, chama o endpoint com token de ADMIN e confere a série (resumido):

```
ResponseEntity<String> resp = call(HttpMethod.GET, "/actuator/prometheus",
        adminToken(), null, String.class);
assertThat(resp.getStatusCode()).isEqualTo(HttpStatus.OK);
assertThat(resp.getBody())
        .contains("servicehub_service_requests_transitions_total{to=\"PENDING\"");
```

A mesma classe prova que `/actuator/health` é público e que um CLIENT recebe 403 em `/actuator/metrics`.

O X-Request-Id

`config/RequestIdFilter` roda antes de todos os outros filtros, inclusive o de segurança, então até um 401 tem id. Trecho real, resumido:

```
@Component
@Order(Ordered.HIGHEST_PRECEDENCE)
public class RequestIdFilter extends OncePerRequestFilter {
    private static final Pattern VALID_ID = Pattern.compile("^[A-Za-z0-9._-]{1,64}$");

    protected void doFilterInternal(...) {
        String received = request.getHeader(HEADER);          // "X-Request-Id"
        String id = (received != null && VALID_ID.matcher(received).matches())
                ? received : UUID.randomUUID().toString();
        MDC.put(MDC_KEY, id);                                  // "requestId"
        response.setHeader(HEADER, id);
        try { chain.doFilter(request, response); }
        finally { MDC.remove(MDC_KEY); }
    }
}
```

Validar o header impede que o cliente injete quebras de linha ou um texto gigante nos seus logs (forjando linhas falsas). O `finally` limpa o MDC antes de a thread voltar ao pool. O contrato do Mapa exige o header em toda resposta, e o CORS o expõe ao navegador.

Logs: texto em desenvolvimento, JSON em produção

Em desenvolvimento, o `application.properties` usa um padrão de texto com o id entre colchetes:

```
logging.pattern.console=%d{HH:mm:ss.SSS} %-5level [%X{requestId:-}] %logger{36} - %msg%n
```

No perfil `prod`, que o Compose ativa com `SPRING_PROFILES_ACTIVE=prod`, o `application-prod.properties` troca para JSON no formato ECS, um objeto por linha, com o `requestId` como campo:

```
logging.structured.format.console=ecs
logging.level.root=INFO
logging.level.org.hibernate.SQL=WARN
```

O SQL sai pelo logger `org.hibernate.SQL`, e não por `show-sql`, justamente para que cada comando SQL carregue o id da requisição que o gerou.

Função de aptidão e ADRs do projeto

`ProtectedRoutesIntegrationTest` percorre todas as rotas protegidas e exige 401 sem token: se alguém criar uma rota e esquecer a segurança, o build falha. É uma função de aptidão de segurança. As decisões do projeto estão em `docs/adr/`, no formato de Nygard:

ADR	Decisão	Por que importa
0001	Stack e versões	Spring Boot 4 e Java 21 fixados por escrito.
0002	Exclusão lógica de usuários e serviços	Apagar não leva junto o histórico de terceiros.

ADR	Decisão	Por que importa
0003	JWT nativo do Spring Security	Sem filtro de token escrito à mão.
0004	ProblemDetail (RFC 9457)	Um único formato de erro em toda a API.
0005	Testcontainers em vez de H2	Os testes rodam as migrações Postgres reais.
0006	Máquina de estados da ordem	A regra central vive como dado no enum; um teste cobre as 25 combinações de estados (5 × 5) para os três papéis.
0007	Admin inicial por variável de ambiente	Nenhuma senha de admin no código.

O guia de estilo do ServiceHub também existe, espalhado pelo Mapa e pelo arquivo de contexto do repositório: `/api/v1`, ProblemDetail, paginação `{content, page}` e `sort` por lista permitida.

14. Construindo com IA

↑ conteúdo

Métricas e logs são código que quase nunca quebra um teste funcional: um erro aqui só aparece em produção, quando o monitoramento cai ou o log vaza um token. Por isso a revisão pesa mais que a geração. Os quatro movimentos:

Movimento	Nesta semana
Especificar	Quero saber quantas ordens chegam a cada estado: um contador por estado de destino, lido só por ADMIN, e um id em toda resposta e todo log.
Gerar	Plano primeiro (onde incrementar, qual tag, qual filtro); depois o código, em passos.
Revisar	Tags com poucos valores, nenhum dado pessoal no log, `/actuator` fechado, header validado, MDC limpo no `finally`.
Provar	Teste que chama `/actuator/prometheus` como ADMIN e acha a série; sem token, 401; a resposta traz `X-Request-Id`.

Prompt fraco e prompt bom

Fraco

```
adiciona métricas e logs na minha api
```

Volta Actuator todo exposto, id de usuário como tag e header sem validar.

Bom

Diz a pergunta de negócio, nomeia métrica e tag, define quem lê, exige header validado e MDC limpo, pede plano e fecha com critério de aceite. É o prompt abaixo.

O prompt desta semana

Continue de onde a Semana 14 parou (CI verde a cada push e PR). Cole no assistente e ajuste ao seu repositório, se algo for diferente:

```
Contexto: API ServiceHub em Java 21 e Spring Boot 4.1, com Spring
Security 7 (JWT, papéis CLIENT, PROVIDER e ADMIN), PostgreSQL 16 e
testes de integração com Testcontainers. A ordem de serviço muda de
estado em ServiceRequestService (PENDING, ACCEPTED, IN_PROGRESS,
COMPLETED, CANCELLED). Já temos spring-boot-starter-actuator.

Objetivo:
1. Uma métrica de negócio: contador Micrometer
   "servicehub.service_requests.transitions" com a tag "to" (estado
   de destino), incrementado ao criar a ordem (PENDING) e a cada
   mudança de status válida, só depois de gravar.
2. Um filtro que garanta o header X-Request-Id em toda resposta e
   coloque o mesmo id no MDC (chave "requestId").
```

```
Restrições:
- Adicionar micrometer-registry-prometheus; expor na web só
  health, info, metrics e prometheus. NADA de include=*.
- /actuator/health e /actuator/info públicos; o resto de /actuator/**
  só para ADMIN, na configuração de segurança existente.
- A tag "to" só pode ter os 5 valores do enum. NÃO usar id de
  usuário, de ordem ou de requisição como tag.
- MeterRegistry injetado pelo construtor; nada de static.
- Filtro com a maior precedência; aceitar o id do cliente só se
  casar com ^[A-Za-z0-9._-]{1,64}$, senão gerar um UUID; remover do
  MDC num finally.
- NÃO logar senha, token nem corpo de requisição; NÃO usar
  System.out; NÃO adicionar biblioteca de log JSON.
- Incluir o requestId no padrão de log do console.

Antes do código: mostre um plano com os arquivos que vai alterar e
onde exatamente o contador é incrementado. Espere minha confirmação.

Critério de aceite: a série aparece em /actuator/prometheus com token
de ADMIN e o header volta na resposta. Escreva testes de integração
que provem isso, e que sem token /actuator/prometheus responde 401.
```

O que a IA costuma errar nesta etapa

- **Tag sem limite:** `userId` ou `requestId` como tag cria uma série por valor e afoga o Prometheus. Funciona no teste com três usuários e só derruba o monitoramento em produção.
- **Actuator escancarado:** `exposure.include=*` e `/actuator/**` público expõem `env` e `heapdump`.
- **Segredo no log:** logar o corpo do login, o token JWT ou o e-mail inteiro "para depurar".
- **Header sem validar:** copiar o `X-Request-Id` do cliente direto para o log permite forjar linhas.
- **MDC sem `finally`**, e incremento do contador antes de gravar (conta transições que falharam).

Checklist de revisão

☐ A tag tem um conjunto pequeno e fixo de valores?

☐ A lista de exposição tem só os quatro endpoints, e só health e info são públicos?

☐ O header recebido é validado e o MDC é limpo num `finally`?

☐ O contador é incrementado depois de a mudança ser gravada?

Prova

Rode `./mvnw verify` com os testes novos. Depois, com a API no ar, crie uma ordem e confira à mão:

```
curl -H "Authorization: Bearer <token de ADMIN>" http://localhost:8090/actuator/prometheus
  → contém servicehub_service_requests_transitions_total{to="PENDING"   (sem token: 401)
curl -i -H "X-Request-Id: teste-123" http://localhost:8090/api/v1/info
  → a resposta traz X-Request-Id: teste-123, e o id aparece nos logs
```

15. Glossário

↑ conteúdo

Observabilidade	Capacidade de entender o estado interno do sistema pelos dados que ele emite.
Monitoramento	Acompanhamento de sinais definidos antes, com alertas por limite.
Métrica	Número agregado ao longo do tempo, como uma contagem ou uma duração.
Contador	Métrica que só cresce: quantas vezes algo aconteceu.
Gauge	Métrica que sobe e desce: o valor no instante da leitura.
Histograma	Distribuição de valores em faixas, base para percentis.
Percentil (p95)	Valor abaixo do qual ficam 95% das observações.
Cardinalidade	Número de valores distintos de uma tag; alta cardinalidade multiplica as séries.
SLI / SLO / SLA	Indicador medido / meta interna para ele / acordo com o cliente, com consequência.
Log estruturado	Evento registrado como pares chave-valor, em geral JSON.
Trace / span	Viagem completa de uma requisição / uma etapa dessa viagem.
MDC	Mapa por thread cujo conteúdo o log inclui em cada linha.
Característica de arquitetura	Requisito fora do domínio que influencia a estrutura e é crítico para o sucesso.
Função de aptidão	Verificação automática e objetiva de uma característica de arquitetura.
ADR	Registro curto de uma decisão de arquitetura: contexto, decisão, consequências.
API Gateway	Ponto único de entrada que roteia, autentica e limita a taxa de requisições.

16. Exercícios

↑ conteúdo

1. Explique a diferença entre monitoramento e observabilidade usando o caso do erro das 14h relatado por um cliente.
2. A API ficou lenta. Em qual pilar você procuraria primeiro, e qual usaria em seguida? Justifique pelo custo e pelo tipo de pergunta que cada um responde.
3. Classifique como contador, gauge ou histograma: (a) avaliações publicadas; (b) ordens em IN_PROGRESS agora; (c) tempo de resposta de `POST /auth/login`; (d) conexões em uso no pool do banco.
4. Um serviço tem tempo médio de 120 ms, mas os usuários reclamam de lentidão. Que métrica você pediria e por quê?
5. Proponha um SLI e um SLO para a listagem pública `GET /api/v1/services`. Explique o orçamento de erro do seu SLO.

6. Um colega quer a tag `clientId` no contador de transições "para saber quem cancela mais". Qual é o problema e como obter essa informação de outro jeito?
7. Leia o `RequestIdFilter` da seção 13. O que aconteceria se (a) o header do cliente não fosse validado; (b) o `MDC.remove` não estivesse num `finally`?
8. Escolha três características de arquitetura para o ServiceHub, justifique cada uma e proponha uma medida ou função de aptidão para uma delas.
9. Escreva um ADR curto (formato de Nygard) para a decisão "logs em JSON no perfil de produção".
10. Que funções um API Gateway assumiria se o ServiceHub fosse dividido em vários serviços? Qual regra deve continuar no serviço, e por quê?
11. Escreva um prompt que peça à IA um contador de avaliações publicadas por nota (1 a 5), com pelo menos três restrições e um critério de aceite verificável.

17. Para estudar

↑ conteúdo

Leituras disponíveis na Minha Biblioteca, pelo acesso institucional:

- RICHARDS, M.; FORD, N. *Fundamentos da arquitetura de software*. Alta Books, 2024. Capítulos 4 a 6 (definir, identificar, medir e governar características de arquitetura) e 19 (decisões de arquitetura). Comece pelo 19, que conversa com os ADRs.
- MARTIN, R. C. *Arquitetura limpa*. Alta Books, 2019. Capítulo 15.

Referências abertas:

- Documentação oficial do Spring Boot, seções Actuator, Observability e Structured Logging: `docs.spring.io/spring-boot`.
- Documentação do Prometheus, tipos de métrica: `prometheus.io/docs`.
- NYGARD, M. *Documenting Architecture Decisions*. 2011.
- W3C, *Trace Context*: `w3.org/TR/trace-context`.